

STARGATE COMMUNICATIONS LTD. SELF-HOSTED AI-POWERED IT/NOC SUPPORT PLATFORM A Serious IT/NOC/AI Engineering Portfolio Project Design, Deployment, Integration, Troubleshooting, Security, Performance Optimization, Knowledge Engineering and Operationalization of a Local AI Support Assistant using Ollama, Qwen and Open WebUI Prepared as an internal engineering project and professional portfolio document Organization: Stargate Communications Ltd. Project domain: ISP / NOC / Network Operations / AI Engineering Implementation basis: Actual deployed environment and troubleshooting experience 2026

Document Control

Executive Summary

This project documents the design and implementation of a self-hosted artificial intelligence platform developed for Stargate Communications Ltd. to support IT operations, network operations center (NOC) activities, ISP customer support, technical knowledge retrieval, and controlled network diagnostics.

The platform combines a local large language model (LLM) running through Ollama with Qwen 3.5 9B and a customized Open WebUI deployment. It uses a Stargate-specific Knowledge Base to ground answers in internal technical documentation and provides controlled, read-only diagnostic tools for ping, DNS lookup, TCP port checks, and traceroute. When an internal Knowledge Base answer is unavailable or insufficient, the assistant can use external web search to obtain current public technical information.

A major engineering objective was to avoid treating an LLM as an unrestricted command executor. The diagnostic layer therefore uses explicit functions with validation and timeouts instead of arbitrary shell execution. The project also addressed practical production issues including Docker image customization, Open WebUI source-level troubleshooting, native tool calling, context-window limitations, database and volume backup, role-based access, and performance considerations on CPU-only infrastructure.

The result is a practical AI-assisted NOC platform rather than a simple chatbot. It demonstrates system administration, Linux, Docker, networking, Python, RAG/Knowledge Base engineering, LLM tool calling, security design, troubleshooting, documentation, and operational readiness.

Abstract

Modern ISP and NOC environments contain large quantities of technical information distributed across network documentation, support procedures, equipment references, IP addressing records, troubleshooting guides, and operational experience. This project implements a self-hosted AI assistant that combines local LLM inference, organization-specific knowledge retrieval, controlled network diagnostic functions, and external web research.

The implemented solution uses Ollama as the local model-serving layer, Qwen 3.5 9B as the language model, Docker as the application deployment mechanism, and Open WebUI as the user interface and orchestration layer. A custom Open WebUI image was created to include Linux networking utilities such as ping, traceroute, and DNS utilities. A controlled diagnostic tool was designed to expose specific read-only operations while preventing arbitrary command execution.

Real troubleshooting is included as engineering evidence. A native tool-calling failure was traced to a context limit: a request containing approximately 5,677 prompt tokens exceeded the configured 4,096-token context. Increasing the context capacity to 8,192 resolved the failure and enabled successful end-to-end tool execution. A separate Open WebUI title-generation/memory compatibility issue was traced through application source code and corrected.

The project demonstrates that a local AI assistant can be integrated into operational IT workflows while maintaining a stronger security boundary than unrestricted AI shell access. It also demonstrates the importance of context management, source prioritization, observability, backups, and explicit distinction between internal facts, live diagnostic evidence, and externally researched information.

1. Introduction

Stargate Communications Ltd. operates in an environment where network availability, Internet service quality, customer connectivity, routing, DNS, access networks, and technical support are operationally important. NOC and support personnel routinely need to interpret technical information, troubleshoot connectivity, consult documentation, and execute diagnostic commands.

The purpose of this project is to introduce a locally hosted AI assistant that can act as a technical support layer without requiring internal operational knowledge to be sent to a public LLM service. The assistant is designed around a source-priority model: Stargate's internal Knowledge Base is the primary source for organization-specific information; controlled diagnostic tools provide live evidence; and external web search provides a fallback for information that is current or absent from internal documentation.

1.1 Motivation

- Reduce time required to locate technical support information.
- Make internal ISP/NOC knowledge searchable through natural language.
- Provide controlled network diagnostics through AI tool calling.
- Keep routine LLM inference self-hosted.
- Create a repeatable architecture that can be extended with additional tools and documentation.
- Develop a demonstrable engineering project suitable for professional IT/NOC/AI roles.

2. Problem Statement

Traditional support workflows depend heavily on engineer experience and manual document search. The same diagnostic questions may be repeatedly answered by different support staff, while internal documentation may be distributed across files, notes, configuration references, and troubleshooting guides.

A general-purpose public chatbot can provide useful technical explanations but cannot safely be assumed to know Stargate's internal network information. Conversely, a local LLM without grounding can hallucinate organization-specific facts. The project therefore asks: How can a self-hosted AI assistant provide useful, technically grounded IT/NOC support while maintaining organizational knowledge boundaries, safe diagnostic capabilities, and a practical operational deployment model?

2.1 Engineering Requirements

- Self-hosted inference.
- Stargate-specific Knowledge Base.
- Native AI function calling.
- Read-only diagnostic tools.
- External web fallback for missing/current information.
- Role separation between administrators and support users.
- Persistent data and backup capability.
- Recoverability after container or host failure.
- Reasonable performance on CPU-only infrastructure.
- No unrestricted AI shell execution.

3. Objectives

- Deploy Ollama for local inference.

- Deploy Open WebUI as the AI interface and orchestration layer.
- Integrate Qwen 3.5 9B.
- Create a Stargate-focused Knowledge Base.
- Implement controlled network diagnostics.
- Enable native tool calling.
- Implement internal-first retrieval with web fallback.
- Document and resolve real deployment failures.
- Establish AI tool security boundaries.
- Establish backup and disaster recovery procedures.
- Optimize practical responsiveness.
- Produce a professional engineering artifact suitable for GitHub and job interviews.

4. Scope

The project covers the local AI stack, knowledge engineering, controlled diagnostics, user operation, troubleshooting, backup, security, and documentation. It does not authorize autonomous changes to production routers, routing protocols, VLANs, firewall rules, or other network configuration.

4.1 In Scope

- Linux server deployment
- Ollama model serving
- Open WebUI Docker deployment
- Qwen 3.5 9B integration
- Knowledge Base / RAG workflow
- Custom Open WebUI tools
- Network diagnostic functions
- External web search fallback
- Application troubleshooting
- Backup and recovery
- Performance/context optimization

4.2 Out of Scope

- Autonomous network configuration changes
- Autonomous routing changes
- Customer credential management
- Unrestricted shell execution
- Replacement of NOC engineers or change-control processes

5. System Architecture

The architecture separates model serving, user interaction, knowledge retrieval, diagnostics, and external research.

```
Users | v Open WebUI (:8080) | +-----+ | | v v Knowledge Base Qwen 3.5 9B | | +----+-----+ | | | v v v
| Tools Web Conversation | | Search | | +-----+-----+-----+ | | | ping DNS lookup TCP/traceroute | +-->
```

5.1 Infrastructure

6. Ollama Deployment

Ollama was selected as the local model-serving layer. It exposes the model to the Dockerized Open WebUI instance through host.docker.internal.

```
[Service] Environment="OLLAMA_HOST=0.0.0.0:11434" Environment="OLLAMA_NUM_PARALLEL=1"
Environment="OLLAMA_MAX_LOADED_MODELS=1" Environment="OLLAMA_KEEP_ALIVE=30m" Nice=-5
```

The configuration favors predictable CPU resource use over high parallelism. On CPU-only infrastructure, multiple loaded models or simultaneous generations can cause contention and latency.

7. Open WebUI Deployment and Customization

Open WebUI was deployed in Docker with persistent application data. A custom image was built so the container contained the networking utilities required by the diagnostic tools.

```
FROM ghcr.io/open-webui/open-webui:main USER root RUN apt-get update \ && apt-get install -y
--no-install-recommends \ iputils-ping traceroute dnsutils \ && rm -rf /var/lib/apt/lists/*
```

The customized image was tested with nslookup and ping from inside the container before tool integration.

```
docker run --rm stargate-open-webui:latest nslookup google.com
```

```
docker exec open-webui ping -4 -c 2 8.8.8.8
```

8. Knowledge Base and RAG Strategy

The Knowledge Base is the primary grounding mechanism for Stargate-specific information. It contains documentation relevant to ISP support, network operations, access networks, organizational IP information, and technical procedures.

The information policy is intentionally simple: internal information comes from the Stargate Knowledge Base first. If internal material is insufficient, the assistant may refine the Knowledge Base query once and then use external web search when appropriate. External information must not silently replace internal information.

8.1 Source Priority

Stargate Knowledge Base

Live read-only diagnostic tools

Authoritative external web sources

General technical knowledge

8.2 Why Internal-First Retrieval Matters

A general LLM may know standard networking concepts but cannot safely infer Stargate's actual infrastructure. Internal-first retrieval reduces hallucination risk by grounding organization-specific answers in controlled documents. External fallback prevents the assistant from becoming stale when users ask for current vendor releases, public documentation, or other changing information.

9. AI Tool Calling and Network Diagnostics

A core engineering feature is controlled tool execution. The model requests specific diagnostic functions rather than arbitrary shell commands.

Stargate Network Diagnostics | `+++ ping(hostname) +++ dns_lookup(hostname) +++ check_tcp_port(hostname, port) +++ traceroute(hostname)`

The tool design uses validation, explicit command construction, timeouts, and fixed operation types. It intentionally does not expose a generic `run_command()` function.

9.1 Ping

`ping -4 -c 4 -W 5 <target>`

9.2 DNS Lookup

`nslookup -type=A -timeout=5 -retry=1 <hostname>`

9.3 TCP Port Check

A Python socket connection is used with a five-second timeout and a valid port range of 1–65535.

9.4 Traceroute

`traceroute -4 -n -m 15 -q 1 -w 2 <target>`

10. Native Tool-Calling Validation

A direct Ollama API test demonstrated that Qwen 3.5 9B can produce a native tool call. The model generated a structured function call requesting ping for 8.8.8.8.

`tool_calls: function: name: ping arguments: hostname: 8.8.8.8`

The diagnostic function was then executed directly from the Open WebUI tool database and produced a real Linux ping result. This established that the model's tool-call capability and the underlying tool implementation were functional.

11. Troubleshooting Case Study: Context-Window Failure

During the first end-to-end test, a ping request failed. Logs revealed the actual error:

HTTP 400 request (5677 tokens) exceeds the available context size (4096 tokens)

The initial symptom suggested a tool or model problem, but the root cause was context capacity. The request included conversation content, system prompt, tool definitions, and Open WebUI context. The combined prompt exceeded 4,096 tokens.

The context capacity was increased to 8,192 tokens. After the change, native tool execution completed successfully.

11.1 Engineering Lesson

LLM tool calling must be evaluated as a complete system. Tool schemas, system prompts, retrieval context, conversation history, and model context capacity all affect successful execution.

12. Troubleshooting Case Study: Open WebUI Title/Memory Error

Open WebUI initially produced a background title-generation error involving a missing model value. Source inspection identified the title-generation path and a memory-review function that assumed a dictionary-like model object. A compatibility adjustment that attempted to use `form_data.get('model')` produced a second error because the model field was a string.

The final adjustment made the memory capability check accept a model identifier string as valid background-task input. The code was compiled with Python and the container was restarted. Subsequent log checks showed no recurrence of the relevant title-generation/model `AttributeError`.

This case demonstrates practical application troubleshooting: reproduce, inspect logs, locate the source path, test the hypothesis, correct the type assumption, validate syntax, restart, and observe logs.

13. System Prompt and Agent Behavior

The system prompt was intentionally reduced after testing showed that an overly elaborate prompt made the assistant slower and less responsive. The final policy establishes only the high-value rules: Knowledge Base first, read-only tools when live evidence is needed, web search only when necessary, no invented Stargate facts, no fabricated tool results, and concise answers.

This optimization illustrates an LLM engineering principle: system prompts are part of the model input context. Excessive policy text increases prompt overhead and can reduce responsiveness, especially on CPU-only inference.

14. Security Architecture

The project treats AI tool execution as a security boundary. The model is not granted general shell access. Each operation has a narrow purpose and explicit validation.

14.1 Least-Privilege Principle

The design follows least privilege: the AI can obtain diagnostic evidence but cannot directly modify network infrastructure. Any future write-capable integration should require explicit authorization, a restricted account, an allowlist, logging, approval workflow, and change control.

15. User and Operational Model

The intended operational model separates administration from support usage. Administrators maintain models, tools, Knowledge Base content, and system configuration. Support users consume approved tools and knowledge without unrestricted administrative capability. This separation supports the transition from personal experiment to operational service.

16. Backup and Disaster Recovery

A complete backup must preserve more than the SQLite database. The recoverable system includes the Open WebUI persistent volume, database, customized Docker image, Docker configuration, custom tools, system prompt, Knowledge Base files, and Ollama service configuration.

Recommended backup set: openwebui-data.tar.gz webui.db stargate-open-webui-image.tar.gz Docker configuration custom tools system prompt Knowledge Base documents Ollama systemd configuration model inventory checksums

A cold backup is preferred for the database and persistent application data. Restoration should be tested rather than assuming that a successful archive implies a recoverable application.

17. Performance Engineering

The system currently runs without a GPU, so CPU inference speed is a practical constraint. Configuration choices were made to reduce contention: one parallel generation, one loaded model at a time, a controlled keep-alive period, and a higher process priority.

The context size was increased from 4,096 to 8,192 only after an actual tool-calling failure demonstrated the need. This is preferable to selecting a very large context by default because larger contexts can increase memory and processing overhead.

17.1 Main Performance Factors

- CPU inference speed

- System-prompt length
- Conversation history
- Knowledge Base retrieval context
- Tool schema size
- Web-search/fetch operations
- Number of simultaneous users
- Model size and quantization

18. Testing and Validation

18.1 Representative Test Questions

- What IP blocks belong to Stargate?
- How should I troubleshoot a customer reporting packet loss?
- Ping 8.8.8.8.
- Do a DNS lookup for google.com.
- Check TCP port 443 on google.com.
- What is the latest RouterOS version?
- What are Stargate's IP blocks and what is their current RPKI status?

19. Operational Troubleshooting Methodology

Observe the user-visible symptom.

Check application/container/service logs.

Identify whether the failure is model, context, tool, network, database, or UI related.

Reproduce with the smallest possible test.

Inspect the relevant configuration or source path.

Make the smallest safe correction.

Validate syntax where applicable.

Restart only the affected component.

Repeat the original test.

Record root cause and corrective action.

20. Results

The implemented system successfully demonstrated local Qwen inference through Ollama, Dockerized Open WebUI operation, customized container networking utilities, Stargate Knowledge Base retrieval, native tool-call generation, execution of a controlled ping diagnostic, and recovery from a context-window limitation. The title-generation/memory compatibility issue was also investigated and corrected.

The principal engineering objective was achieved: an internal AI assistant capable of combining organizational knowledge with controlled live diagnostics while retaining an external research path for information not present in internal documentation.

21. Limitations

- CPU-only inference limits generation speed and concurrent capacity.
- A 9B-class model may be less capable than larger models on complex reasoning.
- Knowledge Base quality depends on document quality and maintenance.
- Web-search results depend on external availability and source quality.
- Tool outputs require correct interpretation.
- Current deployment does not provide autonomous network changes.
- High availability and multi-node inference are not part of the current implementation.

22. Future Development

- GPU acceleration.
- Version-pinned production deployment.
- Centralized logging and metrics.
- Prometheus/Grafana monitoring.
- Tool execution audit logs.
- More structured network diagnostics.
- Approved read-only RouterOS/API integration.
- Ticketing-system integration.
- Automated Knowledge Base ingestion and document versioning.
- Answer-quality and hallucination evaluation dataset.
- Role-based tool authorization.
- High-availability inference architecture.
- Formal disaster-recovery exercises.

23. Professional IT/NOC/AI Engineering Value

24. GitHub Publication Strategy

The public GitHub repository should demonstrate the engineering approach without exposing Stargate confidential information. Real credentials, customer information, production databases, private configuration, and sensitive internal topology must never be committed.

stargate-ai-network-support/ ■■■■ README.md ■■■■ docs/ ■■■■ deployment/ ■■■■ ollama/ ■■■■ open-webui/ ■■■■ system-prompt.txt ■■■■ tools/ ■■■■ knowledge-base/ ■■■■ examples/ ■■■■ scripts/ ■■■■ tests/ ■■■■ screenshots/

Sanitized examples should be used for public Knowledge Base documents. The repository should include an explicit warning that production configuration and private data have been removed or replaced.

25. Recommended GitHub Project Summary

Self-Hosted AI Network & ISP Support Assistant — a production-oriented AI engineering project for Stargate Communications Ltd. combining Ollama, Qwen 3.5 9B, Open WebUI, RAG/Knowledge Base retrieval, native function calling, controlled network diagnostics, external web research, Docker, Linux, and operational troubleshooting.

The project demonstrates how an ISP/NOC organization can deploy a locally hosted AI assistant that is grounded in internal documentation while retaining controlled access to live diagnostic evidence and current public information.

26. Conclusion

This project demonstrates the practical engineering required to move from a local LLM experiment to an operationally useful AI support platform. The key achievement is not installation of a model or web interface in isolation; it is the integration of model serving, knowledge grounding, controlled tools, security boundaries, troubleshooting, performance tuning, and operational recovery.

The architecture provides a foundation for AI-assisted NOC and ISP support while keeping sensitive operational knowledge within the organization's controlled environment. It also establishes a path toward GPU acceleration, structured monitoring, ticket integration, controlled network APIs, and automated evaluation.

As a professional portfolio project, it demonstrates cross-domain capability in Linux administration, Docker, networking, Python, AI/LLM operations, RAG, security, troubleshooting, documentation, and production engineering.

Appendix A — Key Engineering Commands

```
systemctl status ollama curl -s http://127.0.0.1:11434/api/ps docker inspect open-webui docker logs --since 5m
open-webui docker exec open-webui ping -4 -c 2 8.8.8.8 docker exec open-webui nslookup google.com docker exec
-i open-webui python3 - <<'PY' import sqlite3 db="/app/backend/data/webui.db" con=sqlite3.connect(db)
print(con.execute("PRAGMA integrity_check;").fetchone()[0]) con.close() PY python3 -m py_compile <tool-file>.py
```

Appendix B — Production Safety Checklist

- Remove secrets before publishing.
- Use a version-pinned Open WebUI image for production.
- Back up persistent Open WebUI data.
- Back up custom tools and system prompts.
- Version Knowledge Base source documents where appropriate.
- Restrict administrator access.
- Review every AI tool before enabling it.
- Do not expose arbitrary shell execution.
- Keep diagnostic functions read-only.
- Monitor CPU, RAM, disk, containers, and response time.
- Test restoration periodically.
- Document changes to the AI platform.

Appendix C — Suggested Interview Demonstration

Show the Open WebUI interface.

Ask a Stargate-specific Knowledge Base question.

Ask Qwen to ping 8.8.8.8 and demonstrate native tool execution.

Ask a current technical question requiring web fallback.

Explain the 4,096-to-8,192 context troubleshooting case.

Show the restricted diagnostic tool and explain why arbitrary shell execution was avoided.

Explain backup and recovery.

Discuss future GPU acceleration and monitoring.

References and Documentation Sources

The implementation content in this document is based primarily on the actual Stargate engineering environment, deployment work, troubleshooting observations, and configuration developed during the project. Current external references should be added to the final public version using official documentation for the exact software versions deployed.

Open WebUI documentation: <https://docs.openwebui.com/>

Ollama documentation: <https://docs.ollama.com/>

For a public GitHub version, pin exact software versions and cite the official documentation corresponding to those versions.

Confidentiality and Publication Note

Before external publication, all organization-sensitive information must be reviewed. Real credentials, customer information, secrets, private keys, production database files, unrestricted internal configurations, and other confidential material must be removed or replaced with sanitized examples.

Item	Details
Project	Stargate Self-Hosted AI/NOC Support Platform
Organization	Stargate Communications Ltd.
Core platform	Ollama + Qwen 3.5 9B + Open WebUI
Deployment	Self-hosted, Dockerized, CPU-based inference
Knowledge	Internal Knowledge Base / RAG with external web fallback
Tools	Controlled read-only network diagnostics
Audience	NOC engineers, network administrators, support engineers, IT management
Purpose	Technical portfolio, engineering record, thesis-style project documentation
Security	Review and sanitize confidential production information before publication

Component	Implemented/Observed configuration
Server	Dell PowerEdge R740xd
CPU	Intel Xeon Platinum 8173M @ 2.00 GHz
Memory	64 GB RAM
GPU	None; CPU-only inference
Ollama	0.35.0
Model	Qwen 3.5 9B
Embedding	nomic-embed-text
Container	Docker
Open WebUI	Custom image derived from Open WebUI main image
Open WebUI port	8080
Ollama listener	0.0.0.0:11434
Context	8,192 tokens after troubleshooting

Control	Implementation
Arbitrary shell	Not exposed
Target validation	IPv4/hostname validation
Ping	Fixed packet count and timeout
DNS	Fixed options and timeout
TCP	Port validation and socket timeout
Traceroute	Fixed hop count/query count/timeout
Network changes	Not provided
Routing changes	Not provided
Firewall changes	Not provided
Credentials	Excluded from tools and public artifacts

Test	Expected outcome	Purpose
Internal KB question	Grounded internal answer	Verify RAG
Unknown public question	Web fallback	Verify external research
Ping	Actual ping output	Verify tool calling
DNS lookup	Resolver output	Verify DNS tool
TCP check	Open/closed/timeout	Verify socket tool
Traceroute	Hop output or controlled failure	Verify route diagnostic
Invalid target	Validation error	Security
Invalid port	Validation error	Security
Tool timeout	Controlled error	Reliability
Container restart	Data persists	Persistence
Backup/restore	Application recovery	DR
Large prompt	Within context	Capacity

Skill area	Demonstrated work
Linux	Service management, networking utilities, troubleshooting
Docker	Custom image build, container configuration, persistent storage
LLM operations	Ollama deployment, Qwen configuration, context management
Open WebUI	Models, tools, Knowledge Base, source-level troubleshooting
Python	Custom diagnostic functions and validation
Networking	Ping, DNS, TCP, traceroute, ISP/NOC workflows
RAG	Internal Knowledge Base grounding
AI agents	Native function calling and tool execution
Security	Least privilege and restricted diagnostics
Operations	Logging, testing, backup and recovery
Troubleshooting	Root-cause analysis of real failures

Skill area	Demonstrated work
Documentation	Engineering and thesis-style project documentation